Tutorial - 5

Code

```
"""
Tutorial 5: Intelligent Document Search Using Unsupervised Learning
====================================================================
Application : Intelligent search over college notes / research papers.
Objective   : Without any labelled training data,
              - represent documents with word embeddings (GloVe),
              - group them with K-means and hierarchical clustering,
              - discover themes with Latent Dirichlet Allocation (LDA),
              - visualise the collection in 2D with PCA and t-SNE,
              - detect anomalous (out-of-place) documents,
              - retrieve documents for a query by semantic similarity or
                cluster membership, and
              - evaluate retrieval with Precision, Recall, F1, MAP and MRR.

Run with --interactive to type your own queries after the report.
"""

import os
import re
import sys
import gzip
import math
import urllib.request
from collections import Counter
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import nltk
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.decomposition import (PCA, TruncatedSVD, LatentDirichletAllocation,
                                   NMF)
from sklearn.manifold import TSNE
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score, adjusted_rand_score
from scipy.cluster.hierarchy import linkage, dendrogram

# ====================================================================
#  1. Document Collection: 4 subject areas + unrelated documents
#     (the subject is stored only to evaluate clustering at the end;
#      it is never used to build clusters, topics or rankings)
# ====================================================================
DOCUMENTS = {
    # ---- Artificial Intelligence ----
    "D1":  ("AI", "Neural networks and deep learning",
            "Neural networks are models made of layers of artificial neurons. "
            "Deep learning trains networks with many hidden layers using "
            "backpropagation and gradient descent. Activation functions, "
            "dropout and large training datasets help these models learn "
            "complex patterns and achieve high accuracy in prediction tasks."),
    "D2":  ("AI", "Natural language processing",
            "Natural language processing enables computers to understand and "
            "generate human language. Common tasks include tokenization, "
            "part-of-speech tagging, named entity recognition, machine "
            "translation and sentiment analysis. Word embeddings represent the "
            "meaning of words so that algorithms can compare sentences and "
            "answer questions about text."),
    "D8":  ("AI", "Supervised machine learning",
            "Supervised machine learning learns a function from labelled "
            "training examples. Classification and regression algorithms such "
            "as decision trees, logistic regression and support vector machines "
            "predict outputs for new data. Models are evaluated with accuracy, "
            "precision and cross-validation to avoid overfitting."),
    "D9":  ("AI", "Computer vision",
            "Computer vision teaches machines to interpret images and video. "
            "Convolutional neural networks learn visual features to classify "
            "pictures, detect objects, recognise faces and segment medical "
            "scans. Training such models requires large labelled image "
            "datasets and powerful graphics processors."),
    "D10": ("AI", "Reinforcement learning",
            "In reinforcement learning an intelligent agent learns by trial and "
            "error, receiving rewards or penalties from its environment. The "
            "agent improves its policy to maximise long-term reward. This "
            "approach has trained programs to play chess and Go better than "
```

```
        "humans and to control autonomous robots."),
    "D11": ("AI", "Large language models and chatbots",
        "Large language models based on the transformer architecture are "
        "trained on huge text corpora to predict the next word. They "
        "generate fluent answers, write summaries and translate text. "
        "Chatbots and virtual assistants use these models to hold "
        "conversations and understand questions asked in natural language."),
    # ---- Cybersecurity ----
    "D3":  ("Security", "Network security and firewalls",
        "Firewalls filter incoming and outgoing traffic according to "
        "security rules and block unauthorised access. Intrusion detection "
        "and prevention systems monitor networks for suspicious activity, "
        "port scans and attacks. Administrators segment networks and apply "
        "patches to reduce vulnerabilities that hackers could exploit."),
    "D4":  ("Security", "Cryptography and cyberattacks",
        "Cryptography protects confidential information using encryption, "
        "digital signatures and secret keys. Attackers launch cyberattacks "
        "using phishing emails, malware and ransomware to steal sensitive "
        "data or hold it for ransom. Strong encryption keeps stolen data "
        "unreadable to criminals."),
    "D12": ("Security", "Malware and antivirus software",
        "Viruses, worms, spyware and trojans are malicious programs that "
        "infect computers and spread across systems. Antivirus software "
        "scans files for known signatures and suspicious behaviour, then "
        "quarantines or removes threats. Regular updates protect systems "
        "against newly discovered malware."),
    "D13": ("Security", "Authentication and passwords",
        "Authentication verifies the identity of a user before granting "
        "access. Strong passwords, two-factor authentication and biometric "
        "login prevent attackers from hijacking accounts. Systems store "
        "passwords as salted hashes so that a security breach does not "
        "reveal them, and lock accounts after repeated failed attempts."),
    "D14": ("Security", "Virtual private networks and TLS",
        "A virtual private network encrypts traffic between a user and a "
        "remote server, shielding communication from eavesdropping on "
        "public wifi. Transport layer security protects websites with "
        "certificates and encrypted sessions, preventing attackers from "
        "intercepting or tampering with sensitive data."),
    "D15": ("Security", "Denial-of-service attacks",
        "Distributed denial-of-service attacks flood servers with malicious "
        "traffic from botnets of infected machines until the service "
        "crashes. Defences include rate limiting, traffic filtering, "
        "scrubbing centres and redundant infrastructure that keep "
        "organisations online during an attack."),
    # ---- Databases ----
    "D5":  ("Databases", "Relational databases and SQL",
        "Relational databases store data in tables of rows and columns "
        "linked by primary and foreign keys. SQL queries select, join, "
        "insert, update and delete records. A database management system "
        "enforces the schema and constraints so that stored records remain "
        "consistent."),
    "D6":  ("Databases", "Database indexing",
        "Indexes such as B-trees and hash indexes speed up database lookups "
        "by avoiding full table scans. The query optimizer chooses an "
        "execution plan that uses the right index, reducing query response "
        "time. Too many indexes, however, slow down inserts and updates."),
    "D16": ("Databases", "Transactions and recovery",
        "Transactions group database operations so that they satisfy the "
        "ACID properties: atomicity, consistency, isolation and durability. "
        "Locking and concurrency control prevent conflicting updates, while "
        "write-ahead logging allows the database to recover committed "
        "records after a crash."),
    "D17": ("Databases", "Normalization",
        "Normalization organises database tables to reduce redundancy and "
        "update anomalies. Functional dependencies between attributes "
        "guide the decomposition of a schema into first, second, third and "
        "Boyce-Codd normal forms, so that each fact is stored in exactly "
        "one table."),
    "D18": ("Databases", "NoSQL databases",
        "NoSQL databases such as MongoDB and Cassandra store data as "
        "documents, key-value pairs or wide columns instead of relational "
        "tables. They scale horizontally across many servers, replicate "
        "records for availability and handle big data workloads with "
        "flexible schemas and fast queries."),
    # ---- Computer Networking ----
    "D19": ("Networking", "The TCP/IP protocol suite",
        "The TCP/IP protocol suite moves packets of data across the "
        "internet. IP handles addressing and delivers packets between "
        "hosts, while TCP provides reliable, ordered delivery using "
        "acknowledgements, retransmission and congestion control. UDP "
```

```
           "offers faster but unreliable transmission."),
    "D20": ("Networking", "Routing",
           "Routers forward packets between networks using routing tables. "
           "Routing protocols such as OSPF and BGP exchange information about "
           "network topology and compute the best paths for traffic across "
           "the internet, adapting when links fail or become congested."),
    "D21": ("Networking", "Wireless networks",
           "Wifi and cellular networks transmit data over radio waves between "
           "devices and access points. Bandwidth, signal strength, frequency "
           "and interference determine connection speed and latency. Mobile "
           "networks hand over connections between base stations as users "
           "move."),
    "D22": ("Networking", "The OSI model",
           "The OSI reference model divides network communication into seven "
           "layers: physical, data link, network, transport, session, "
           "presentation and application. Each layer provides services to the "
           "layer above, from transmitting bits over cables to the protocols "
           "used by web browsers."),
    "D23": ("Networking", "Domain Name System",
           "The Domain Name System translates website names into IP "
           "addresses so that computers can locate servers on the internet. "
           "DNS resolvers query root, top-level domain and authoritative name "
           "servers, and cache the answers to make later lookups faster."),
    # ---- Unrelated documents accidentally added ----
    "D7":  ("Other", "A document about gardening",
           "Tomatoes, roses and herbs grow best in sunny gardens with rich "
           "compost and well-drained soil. Water the plants early in the "
           "morning, add mulch to keep moisture in, and remove weeds and "
           "pests regularly. Prune flowering shrubs after they bloom."),
    "D24": ("Other", "Football match report",
           "The football team won the championship final after a dramatic "
           "late goal in extra time. The goalkeeper made several brilliant "
           "saves, the striker scored twice, and thousands of fans celebrated "
           "in the stadium after the referee blew the final whistle."),
}

# ======================================================================
#  2. Test queries and relevance judgments (for retrieval evaluation)
# ======================================================================
TEST_QUERIES = {
    "Q1": ("documents about protecting computer networks",
           {"D3", "D4", "D12", "D14", "D15"}),
    "Q2": ("how do computers understand human language", {"D2", "D11"}),
    "Q3": ("making database queries run faster", {"D6", "D5", "D18"}),
    "Q4": ("how data travels across the internet",
           {"D19", "D20", "D21", "D22", "D23"}),
    "Q5": ("teaching machines to learn from examples",
           {"D8", "D1", "D10", "D9"}),
}

TOP_K = 5
SEED = 42
N_CLUSTERS = 4               # documents were gathered from four subject areas
GLOVE_URL = ("https://github.com/RaRe-Technologies/gensim-data/releases/"
             "download/glove-wiki-gigaword-100/glove-wiki-gigaword-100.gz")
CACHE_DIR = os.path.join(os.path.expanduser("~"), ".cache", "ir_tutorials")
GLOVE_WORDS = 100000         # most frequent GloVe words to keep

# ======================================================================
#  3. Preprocessing
# ======================================================================
def load_stopwords():
    try:
        nltk.download('stopwords', quiet=True)
        from nltk.corpus import stopwords
        return set(stopwords.words('english'))
    except Exception:
        from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
        return set(ENGLISH_STOP_WORDS)


STOP_WORDS = load_stopwords() | {"use", "used", "using", "include", "includes",
                                 "document", "documents", "make", "making"}


def tokenize(text):
    return [t for t in re.findall(r'[a-z]+', text.lower())
            if t not in STOP_WORDS and len(t) > 1]


def doc_text(d):
```

```
    _, title, body = DOCUMENTS[d]
    return title + ". " + body


# ============================================================================
#  4. Word Embeddings (pretrained GloVe, 100 dimensions)
# ============================================================================
def load_glove():
    """Return {word: vector} for the most frequent GloVe words, or None."""
    os.makedirs(CACHE_DIR, exist_ok=True)
    cache = os.path.join(CACHE_DIR, f"glove100_top{GLOVE_WORDS}.npz")
    if os.path.exists(cache):
        data = np.load(cache)
        return dict(zip(data["words"].tolist(), data["vectors"]))

    gz_path = os.path.join(CACHE_DIR, "glove-wiki-gigaword-100.gz")
    try:
        if not os.path.exists(gz_path):
            print("Downloading GloVe vectors (~130 MB, first run only)...")
            urllib.request.urlretrieve(GLOVE_URL, gz_path)
        words, vecs = [], []
        with gzip.open(gz_path, "rt", encoding="utf8") as f:
            next(f)                                  # header: "400000 100"
            for i, line in enumerate(f):
                if i >= GLOVE_WORDS:
                    break
                parts = line.rstrip().split(" ")
                words.append(parts[0])
                vecs.append(np.asarray(parts[1:], dtype=np.float32))
        vectors = np.vstack(vecs)
        np.savez(cache, words=np.array(words), vectors=vectors)
        return dict(zip(words, vectors))
    except Exception as e:
        print(f"Could not load GloVe ({e}); falling back to LSA embeddings.")
        return None


def normalise(m):
    norms = np.linalg.norm(m, axis=-1, keepdims=True)
    return m / np.where(norms == 0, 1, norms)


class Embedder:
    """
    SIF document embeddings (Smooth Inverse Frequency, Arora et al. 2017):
      1. document vector = weighted mean of its GloVe word vectors, with
         weight a / (a + p(w)); p(w) is the word's frequency in English,
         estimated from its GloVe frequency rank using Zipf's law, so
         generic words ("data", "use") count less than specific ones,
      2. remove the common component (first singular vector) that all
         documents share,
      3. L2-normalise, so cosine similarity = dot product.
    """

    def __init__(self, doc_ids, glove, a=1e-3):
        self.doc_ids = doc_ids
        self.glove = glove
        self.a = a
        self.rank = {w: i + 1 for i, w in enumerate(glove)}
        self.zipf_norm = math.log(len(glove)) + 0.5772
        self.tokens = {d: tokenize(doc_text(d)) for d in doc_ids}
        N = len(doc_ids)
        df = Counter(t for d in doc_ids for t in set(self.tokens[d]))
        self.idf = {t: math.log((N + 1) / (c + 1)) + 1 for t, c in df.items()}
        self.default_idf = math.log(N + 1) + 1     # query words not in corpus

        # unit word vectors of each document, for word-level matching
        self.doc_words = {}
        for d in doc_ids:
            words = sorted({t for t in self.tokens[d] if t in glove})
            self.doc_words[d] = (words, normalise(np.vstack([glove[w] for w in words])))

        raw = np.vstack([self._average(self.tokens[d]) for d in doc_ids])
        self.raw = normalise(raw)                  # before removing common part
        self.u = np.linalg.svd(raw, full_matrices=False)[2][0]
        self.matrix = normalise(raw - np.outer(raw @ self.u, self.u))

    def sif_weight(self, w):
        p = 1 / (self.rank[w] * self.zipf_norm)
        return self.a / (self.a + p)
```

```
    def _average(self, tokens):
        counts = Counter(t for t in tokens if t in self.glove)
        if not counts:
            return np.zeros(next(iter(self.glove.values())).shape)
        weights = np.array([c * self.sif_weight(t) for t, c in counts.items()])
        vecs = np.vstack([self.glove[t] for t in counts])
        return weights @ vecs / weights.sum()

    def embed_query(self, text):
        v = self._average(tokenize(text))
        return normalise(v - (v @ self.u) * self.u)

    def word_matches(self, query, d):
        """For every query word, the most similar word in document d:
        returns (weight, similarity, query word, document word) tuples."""
        words, W = self.doc_words[d]
        matches = []
        for q in tokenize(query):
            if q not in self.glove:
                continue
            sims = W @ normalise(self.glove[q])
            best = int(np.argmax(sims))
            weight = self.sif_weight(q) * self.idf.get(q, self.default_idf)
            matches.append((weight, float(sims[best]), q, words[best]))
        return matches

    def semantic_score(self, query, d):
        """Weighted mean of each query word's best match in the document."""
        m = self.word_matches(query, d)
        if not m:
            return 0.0
        return sum(w * s for w, s, _, _ in m) / sum(w for w, _, _, _ in m)

    def word_vector(self, w):
        return self.glove.get(w)


class LSAEmbedder:
    """Fallback when GloVe is unavailable: Latent Semantic Analysis."""

    def __init__(self, doc_ids):
        self.doc_ids = doc_ids
        self.glove = None
        self.tokens = {d: tokenize(doc_text(d)) for d in doc_ids}
        self.vectorizer = TfidfVectorizer(tokenizer=tokenize, lowercase=False,
                                          token_pattern=None)
        X = self.vectorizer.fit_transform([doc_text(d) for d in doc_ids])
        self.svd = TruncatedSVD(n_components=15, random_state=SEED).fit(X)
        self.matrix = normalise(self.svd.transform(X))
        self.raw = self.matrix

    def embed_query(self, text):
        q = self.svd.transform(self.vectorizer.transform([text]))[0]
        return normalise(q)

    def word_matches(self, query, d):
        return []

    def semantic_score(self, query, d):
        return float(self.matrix[self.doc_ids.index(d)] @ self.embed_query(query))

    def word_vector(self, w):
        return None


# ============================================================================
#  5. Anomaly Detection (k-nearest-neighbour similarity)
# ============================================================================
def detect_anomalies(doc_ids, E, k=3):
    """
    A document is anomalous if it is far from its k nearest neighbours.
    Score = mean cosine similarity to the k nearest other documents.
    Threshold = median - 3 * MAD (robust to the outliers themselves).
    """
    S = E @ E.T
    np.fill_diagonal(S, -np.inf)
    knn = np.sort(S, axis=1)[:, -k:].mean(axis=1)
    median = np.median(knn)
    mad = np.median(np.abs(knn - median)) * 1.4826
    threshold = median - 3 * mad
    anomalies = [d for d, s in zip(doc_ids, knn) if s < threshold]
```

```
    return dict(zip(doc_ids, knn)), threshold, anomalies


# ============================================================================
#  6. Clustering
# ============================================================================
def choose_k(E, k_range):
    scores = {}
    for k in k_range:
        labels = KMeans(n_clusters=k, n_init=20, random_state=SEED).fit_predict(E)
        scores[k] = silhouette_score(E, labels, metric="cosine")
    return max(scores, key=scores.get), scores


def purity(true, pred):
    total = 0
    for c in set(pred):
        members = [t for t, p in zip(true, pred) if p == c]
        total += Counter(members).most_common(1)[0][1]
    return total / len(true)


def top_terms(all_ids, member_ids, n=6):
    """Terms with the highest mean TF-IDF inside a cluster; IDF is computed
    over the whole collection, so terms shared by every cluster rank low."""
    vec = TfidfVectorizer(tokenizer=tokenize, lowercase=False, token_pattern=None)
    X = vec.fit_transform([doc_text(d) for d in all_ids])
    rows = [all_ids.index(d) for d in member_ids]
    weights = np.asarray(X[rows].mean(axis=0)).ravel()
    terms = vec.get_feature_names_out()
    return [terms[i] for i in weights.argsort()[::-1][:n]]


# ============================================================================
#  7. Retrieval
# ============================================================================
class SearchEngine:
    def __init__(self, embedder, cluster_of, centroids):
        self.emb = embedder
        self.doc_ids = embedder.doc_ids
        self.cluster_of = cluster_of            # doc -> cluster id (or -1)
        self.centroids = centroids              # cluster id -> unit vector
        self.keyword = TfidfVectorizer(tokenizer=tokenize, lowercase=False,
                                       token_pattern=None)
        self.K = self.keyword.fit_transform([doc_text(d) for d in self.doc_ids])

    def semantic(self, query, k=TOP_K):
        """Word-level semantic matching: each query word is matched to the
        closest word (in embedding space) of every document."""
        scores = [(d, self.emb.semantic_score(query, d)) for d in self.doc_ids]
        return sorted(scores, key=lambda x: -x[1])[:k]

    def doc_vector(self, query, k=TOP_K):
        """Cosine similarity between the query and document SIF vectors."""
        sims = self.emb.matrix @ self.emb.embed_query(query)
        order = np.argsort(-sims)[:k]
        return [(self.doc_ids[i], float(sims[i])) for i in order]

    def cluster_based(self, query, k=TOP_K):
        """Pick the cluster whose centroid is closest to the query, return its
        members ranked by semantic score, then fill up from the next clusters."""
        q = self.emb.embed_query(query)
        cluster_rank = sorted(self.centroids, key=lambda c: -self.centroids[c] @ q)
        results = []
        for c in cluster_rank:
            members = [(d, self.emb.semantic_score(query, d))
                       for d in self.doc_ids if self.cluster_of[d] == c]
            results += sorted(members, key=lambda x: -x[1])
            if len(results) >= k:
                break
        return results[:k]

    def keyword_baseline(self, query, k=TOP_K):
        sims = (self.K @ self.keyword.transform([query]).T).toarray().ravel()
        order = [i for i in np.argsort(-sims) if sims[i] > 0][:k]
        return [(self.doc_ids[i], float(sims[i])) for i in order]

    def explain(self, query, d, n=3):
        """Why is d relevant? The query words with the strongest matches."""
        matches = sorted(self.emb.word_matches(query, d), key=lambda m: -m[1])
        if not matches:
```

```
            return "-"
        return ", ".join(f"{q}~{w} ({s:.2f})" for _, s, q, w in matches[:n])


def cosine(a, b):
    return float(a @ b / (np.linalg.norm(a) * np.linalg.norm(b)))


# ======================================================================
#  8. Evaluation metrics
# ======================================================================
def evaluate(ranked, relevant, k=TOP_K):
    hits = [d in relevant for d in ranked[:k]]
    p = sum(hits) / k
    r = sum(hits) / len(relevant)
    f1 = 2 * p * r / (p + r) if p + r else 0.0
    rr = next((1 / i for i, h in enumerate(hits, 1) if h), 0.0)
    found, ap = 0, 0.0
    for i, h in enumerate(hits, 1):
        if h:
            found += 1
            ap += found / i
    return {"P@5": p, "R@5": r, "F1@5": f1, "RR": rr, "AP": ap / len(relevant)}


# ======================================================================
#  9. Display helpers and plots
# ======================================================================
def header(title):
    print("\n" + "=" * 78)
    print(f" {title}")
    print("=" * 78)


def title_of(d):
    return DOCUMENTS[d][1]


def plot_2d(points, doc_ids, cluster_of, anomalies, title, path):
    plt.figure(figsize=(10, 7))
    clusters = sorted(set(cluster_of.values()) - {-1})
    colors = plt.cm.tab10.colors
    for c in clusters:
        idx = [i for i, d in enumerate(doc_ids) if cluster_of[d] == c]
        plt.scatter(points[idx, 0], points[idx, 1], s=90,
                    color=colors[c % 10], label=f"Cluster {c + 1}")
    idx = [i for i, d in enumerate(doc_ids) if d in anomalies]
    if idx:
        plt.scatter(points[idx, 0], points[idx, 1], s=140, marker="X",
                    color="red", label="Anomaly")
    for i, d in enumerate(doc_ids):
        plt.annotate(d, points[i], textcoords="offset points", xytext=(6, 4),
                     fontsize=9)
    plt.title(title)
    plt.legend()
    plt.grid(alpha=0.3)
    plt.savefig(path, bbox_inches="tight")
    plt.close()


# ======================================================================
#  10. Main
# ======================================================================
def main():
    output_dir = os.path.dirname(os.path.abspath(__file__))
    doc_ids = list(DOCUMENTS)
    np.random.seed(SEED)

    header("Tutorial 5: Intelligent Document Search Using Unsupervised Learning")
    print(f"Documents collected: {len(doc_ids)}")
    for d in doc_ids:
        print(f"  {d:<4} {title_of(d)}")

    # ---------- Step 1: Embeddings ----------
    header("Step 1: Document Embeddings")
    glove = load_glove()
    if glove is not None:
        emb = Embedder(doc_ids, glove)
        print(f"Model: GloVe (Wikipedia + Gigaword), {len(glove)} words x "
              f"{emb.matrix.shape[1]} dimensions")
        print("Document vector = SIF-weighted mean of word vectors, common "
```

```
            "component removed, L2-normalised")
        covered = sum(t in glove for d in doc_ids for t in emb.tokens[d])
        total = sum(len(emb.tokens[d]) for d in doc_ids)
        print(f"Vocabulary coverage: {covered}/{total} tokens "
              f"({100 * covered / total:.1f}%)")
        print("\nWord similarity examples (cosine of GloVe vectors):")
        for a, b in [("protecting", "security"), ("protecting", "firewall"),
                     ("networks", "internet"), ("database", "sql"),
                     ("learning", "neural"), ("gardening", "encryption")]:
            print(f"  {a:<11} ~ {b:<11} {cosine(glove[a], glove[b]):.3f}")
    else:
        emb = LSAEmbedder(doc_ids)
        print(f"Model: LSA (TF-IDF + truncated SVD), {emb.matrix.shape[1]} dimensions")
    E = emb.matrix
    print(f"\nEmbedding matrix shape: {E.shape}   (first 6 dimensions shown)")
    for i, d in enumerate(doc_ids):
        print(f"  {d:<4} [" + ", ".join(f"{v:+.3f}" for v in E[i, :6]) + ", ...]")

    # ---------- Step 2: Anomaly detection ----------
    header("Step 2: Anomaly Detection (mean similarity to 3 nearest neighbours)")
    print("Computed on the document vectors before the shared component is "
          "removed,\nso a document unlike the rest of the collection stands out.")
    scores, threshold, anomalies = detect_anomalies(doc_ids, emb.raw)
    print(f"\nMedian score = {np.median(list(scores.values())):.3f}. Lowest scores:")
    for d in sorted(doc_ids, key=scores.get)[:6]:
        flag = "  <-- ANOMALY" if d in anomalies else ""
        print(f"  {d:<4} {scores[d]:.3f}  {title_of(d)}{flag}")
    print(f"  ... (remaining documents score higher)")
    print(f"Threshold (median - 3*MAD) = {threshold:.3f}")
    print(f"Detected anomalies: {anomalies}")

    # ---------- Step 3: Clustering ----------
    header("Step 3: Clustering (anomalies excluded)")
    clean = [d for d in doc_ids if d not in anomalies]
    Ec = E[[doc_ids.index(d) for d in clean]]
    k = N_CLUSTERS
    _, sil = choose_k(Ec, range(2, 9))
    print("Silhouette score (cosine) of K-means for different k (for reference):")
    for kk, s in sil.items():
        print(f"  k = {kk}: {s:.4f}" + ("   <-- used" if kk == k else ""))
    print(f"Silhouette scores are close for k = 3-5 on so few documents, so k = {k}\n"
          f"is chosen to match the number of subject areas the notes come from.")

    km = KMeans(n_clusters=k, n_init=50, random_state=SEED).fit(Ec)
    km_labels = km.labels_
    hc_labels = AgglomerativeClustering(n_clusters=k, metric="cosine",
                                        linkage="average").fit_predict(Ec)
    cluster_of = {d: int(c) for d, c in zip(clean, km_labels)}
    cluster_of.update({d: -1 for d in anomalies})
    centroids = {c: normalise(Ec[km_labels == c].mean(axis=0)) for c in range(k)}

    true = [DOCUMENTS[d][0] for d in clean]
    print(f"\nK-means vs. hierarchical (average linkage) agreement, "
          f"ARI = {adjusted_rand_score(km_labels, hc_labels):.3f}")
    print("Agreement with the true subject areas (used only for evaluation):")
    print(f"  K-means      : purity = {purity(true, km_labels):.3f}, "
          f"ARI = {adjusted_rand_score(true, km_labels):.3f}")
    print(f"  Hierarchical : purity = {purity(true, hc_labels):.3f}, "
          f"ARI = {adjusted_rand_score(true, hc_labels):.3f}")

    # ---------- Step 4: Topic modelling (LDA) ----------
    header(f"Step 4: Topic Modelling ({k} topics)")
    texts = [doc_text(d) for d in clean]
    topic_models = {}

    # LDA: probabilistic model over word counts (words in >= 2 documents)
    cv = CountVectorizer(tokenizer=tokenize, lowercase=False, token_pattern=None,
                         min_df=2)
    lda = LatentDirichletAllocation(n_components=k, learning_method="batch",
                                    max_iter=500, doc_topic_prior=0.1,
                                    topic_word_prior=0.05, random_state=SEED)
    topic_models["LDA"] = (lda.fit_transform(cv.fit_transform(texts)),
                           lda.components_, cv.get_feature_names_out())

    # NMF: factorises the TF-IDF matrix into document-topic x topic-word
    tv = TfidfVectorizer(tokenizer=tokenize, lowercase=False, token_pattern=None)
    nmf = NMF(n_components=k, init="nndsvd", max_iter=1000, random_state=SEED)
    topic_models["NMF"] = (nmf.fit_transform(tv.fit_transform(texts)),
                           nmf.components_, tv.get_feature_names_out())
```

```
agreement = {}
for name, (doc_topics, components, vocab) in topic_models.items():
    agreement[name] = adjusted_rand_score(km_labels, doc_topics.argmax(axis=1))
    print(f"\n{name} topics (agreement of dominant topic with K-means "
          f"clusters, ARI = {agreement[name]:.3f}):")
    for t, comp in enumerate(components):
        words = [vocab[i] for i in comp.argsort()[::-1][:8]]
        print(f"  Topic {t + 1}: {', '.join(words)}")

chosen = max(agreement, key=agreement.get)
doc_topics, components, vocab = topic_models[chosen]
topic_words = {t: [vocab[i] for i in comp.argsort()[::-1][:8]]
               for t, comp in enumerate(components)}
dominant = {d: int(doc_topics[i].argmax()) for i, d in enumerate(clean)}
print(f"\nLDA needs many more words than {len(clean)} short documents provide, "
      f"so its topics\nare unstable here. Cluster summaries below use {chosen} "
      f"topics, which agree\nbetter with the clusters.")

# ---------- Step 5: Cluster summaries ----------
header("Step 5: Automatically Generated Clusters and Topic Summaries")
for c in range(k):
    members = [d for d in clean if cluster_of[d] == c]
    idx = [doc_ids.index(d) for d in members]
    rep = members[int(np.argmax(E[idx] @ centroids[c]))]
    topic = Counter(dominant[d] for d in members).most_common(1)[0][0]
    print(f"\nCluster {c + 1} ({len(members)} documents): {', '.join(members)}")
    for d in members:
        print(f"    {d:<4} {title_of(d)}")
    print(f"  Key terms      : {', '.join(top_terms(clean, members))}")
    print(f"  Dominant topic : {chosen} Topic {topic + 1} "
          f"({', '.join(topic_words[topic][:5])})")
    print(f"  Representative : {rep} - {title_of(rep)}")
if anomalies:
    print(f"\nUnclustered anomalies: "
          + ", ".join(f"{d} ({title_of(d)})" for d in anomalies))
    for d in anomalies:
        v = E[doc_ids.index(d)]
        best = max(centroids, key=lambda c: centroids[c] @ v)
        print(f"  {d}: best-fitting cluster is {best + 1} with similarity "
              f"{centroids[best] @ v:.3f} (members average "
              f"{np.mean(Ec[km_labels == best] @ centroids[best]):.3f})")

# ---------- Step 6: Dimensionality reduction ----------
header("Step 6: 2D Visualisation (PCA and t-SNE)")
pca = PCA(n_components=2, random_state=SEED)
P = pca.fit_transform(emb.raw)
print("PCA of the raw document vectors (the space used for anomaly detection):")
print(f"  explained variance PC1 = {pca.explained_variance_ratio_[0]:.3f}, "
      f"PC2 = {pca.explained_variance_ratio_[1]:.3f}")
print("  PC1 separates computing documents from the unrelated ones.")
T = TSNE(n_components=2, perplexity=5, init="pca", metric="cosine",
         random_state=SEED).fit_transform(E)
print("t-SNE of the clustering vectors (shared component removed) shows the "
      "clusters.")
plot_2d(P, doc_ids, cluster_of, anomalies,
        "Documents in 2D (PCA of raw vectors): anomalies stand apart",
        os.path.join(output_dir, "pca_clusters.png"))
plot_2d(T, doc_ids, cluster_of, anomalies,
        "Documents in 2D (t-SNE of clustering vectors)",
        os.path.join(output_dir, "tsne_clusters.png"))

plt.figure(figsize=(12, 6))
dendrogram(linkage(E, method="average", metric="cosine"),
           labels=[f"{d}" for d in doc_ids], leaf_rotation=90)
plt.title("Hierarchical Clustering Dendrogram (average linkage, cosine)")
plt.ylabel("Cosine distance")
plt.savefig(os.path.join(output_dir, "dendrogram.png"), bbox_inches="tight")
plt.close()
print("Saved: pca_clusters.png, tsne_clusters.png, dendrogram.png")

# ---------- Step 7: Retrieval ----------
engine = SearchEngine(emb, cluster_of, centroids)
query = TEST_QUERIES["Q1"][0]
header(f'Step 7: Search Results for "{query}"')
print(f"Query words: {tokenize(query)}")
q = emb.embed_query(query)
best_c = max(centroids, key=lambda c: centroids[c] @ q)
print(f"Closest cluster: Cluster {best_c + 1} "
      f"(similarity {centroids[best_c] @ q:.3f}) -> searched first")
print(f"\nCluster-based semantic search:")
```

```
    print(f"{'Rank':<5}{'Doc':<5}{'Score':>7}  {'Cluster':<8} Title / Why it matches")
    print("-" * 78)
    for i, (d, s) in enumerate(engine.cluster_based(query), 1):
        c = cluster_of[d]
        print(f"{i:<5}{d:<5}{s:>7.3f}  {('C' + str(c + 1)) if c >= 0 else 'anom':<8} "
              f"{title_of(d)}")
        print(f"{'':<27}matches: {engine.explain(query, d)}")
    print("Score = weighted mean similarity between each query word and its "
          "closest\nword in the document (shown as query~document (cosine)).")
    no_overlap = [d for d, _ in engine.cluster_based(query)
                  if not set(tokenize(query)) & set(emb.tokens[d])]
    if no_overlap:
        print(f"Retrieved without containing any exact query word: "
              f"{', '.join(no_overlap)}")

    for name, fn in [("Semantic search over all documents", engine.semantic),
                     ("Keyword (TF-IDF) baseline", engine.keyword_baseline)]:
        print(f"\n{name}:")
        for i, (d, s) in enumerate(fn(query), 1):
            print(f"  {i}. {d:<4} {s:.3f}  {title_of(d)}")

    # ---------- Step 8: Evaluation ----------
    header("Step 8: Retrieval Evaluation (top 5)")
    methods = {"Semantic": engine.semantic,
               "Doc-vector": engine.doc_vector,
               "Cluster-based": engine.cluster_based,
               "Keyword TF-IDF": engine.keyword_baseline}
    summary = {m: [] for m in methods}
    for qid, (text, rel) in TEST_QUERIES.items():
        print(f'\n{qid}: "{text}"   relevant = {sorted(rel, key=lambda x: int(x[1:]))}')
        for m, fn in methods.items():
            ranked = [d for d, _ in fn(text)]
            res = evaluate(ranked, rel)
            summary[m].append(res)
            print(f"  {m:<15} {ranked}  P@5 = {res['P@5']:.2f}  "
                  f"R@5 = {res['R@5']:.2f}  AP = {res['AP']:.2f}")

    print(f"\n{'Method':<16}{'P@5':>8}{'R@5':>8}{'F1@5':>8}{'MRR':>8}{'MAP':>8}")
    print("-" * 56)
    means = {}
    for m, results in summary.items():
        avg = {key: np.mean([r[key] for r in results]) for key in results[0]}
        means[m] = avg
        print(f"{m:<16}{avg['P@5']:>8.3f}{avg['R@5']:>8.3f}{avg['F1@5']:>8.3f}"
              f"{avg['RR']:>8.3f}{avg['AP']:>8.3f}")

    best = max(means, key=lambda m: means[m]["AP"])
    header("Summary")
    print(f"""* Embeddings: GloVe word vectors combined into SIF document vectors let
  the system match related words (protecting~protect, networks~systems).
* Anomalies: {', '.join(f'{d} ({title_of(d)})' for d in anomalies) or 'none'}
  were flagged because they are far from every other document.
* Clustering: K-means found {k} clusters with purity {purity(true, km_labels):.2f}
  against the true subjects. Misplaced documents share vocabulary with
  another subject, e.g. DNS (servers, query, cache) with databases.
* Topics: NMF gave clear themes, but LDA was unstable on a corpus this small.
* Retrieval: {best} search had the best MAP ({means[best]['AP']:.3f}) vs.
  {means['Keyword TF-IDF']['AP']:.3f} for keyword TF-IDF. Restricting the search to the
  closest cluster removes off-topic documents that share a single word
  with the query (e.g. "Computer vision" for "computer networks").""")

    # ---------- Optional interactive search ----------
    if "--interactive" in sys.argv:
        header("Interactive Search (press Enter on an empty line to quit)")
        while True:
            try:
                text = input("\nQuery> ").strip()
            except EOFError:
                break
            if not text:
                break
            for i, (d, s) in enumerate(engine.semantic(text), 1):
                print(f"  {i}. {d:<4} {s:.3f}  {title_of(d)}")
                print(f"       matches: {engine.explain(text, d)}")


if __name__ == '__main__':
    main()
```

Output

```
(.venv) PS D:\SVNIT\Semester - 7\Information Retrieval\Tutorials\Tutorial - 5> python main.py

================================================================================
 Tutorial 5: Intelligent Document Search Using Unsupervised Learning
================================================================================
Documents collected: 24
  D1   Neural networks and deep learning
  D2   Natural language processing
  D8   Supervised machine learning
  D9   Computer vision
  D10  Reinforcement learning
  D11  Large language models and chatbots
  D3   Network security and firewalls
  D4   Cryptography and cyberattacks
  D12  Malware and antivirus software
  D13  Authentication and passwords
  D14  Virtual private networks and TLS
  D15  Denial-of-service attacks
  D5   Relational databases and SQL
  D6   Database indexing
  D16  Transactions and recovery
  D17  Normalization
  D18  NoSQL databases
  D19  The TCP/IP protocol suite
  D20  Routing
  D21  Wireless networks
  D22  The OSI model
  D23  Domain Name System
  D7   A document about gardening
  D24  Football match report

================================================================================
 Step 1: Document Embeddings
================================================================================
Model: GloVe (Wikipedia + Gigaword), 100000 words x 100 dimensions
Document vector = SIF-weighted mean of word vectors, common component removed, L2-normalised
Vocabulary coverage: 724/748 tokens (96.8%)

Word similarity examples (cosine of GloVe vectors):
  protecting  ~ security    0.545
  protecting  ~ firewall    0.246
  networks    ~ internet    0.724
  database    ~ sql         0.543
  learning    ~ neural      0.392
  gardening   ~ encryption  0.085

Embedding matrix shape: (24, 100)   (first 6 dimensions shown)
  D1   [+0.010, +0.067, -0.161, +0.125, -0.096, -0.221, ...]
  D2   [+0.077, +0.175, +0.043, +0.108, +0.004, +0.018, ...]
  D8   [-0.101, +0.145, -0.218, +0.234, -0.138, -0.056, ...]
  D9   [+0.026, +0.103, -0.058, +0.056, +0.042, -0.022, ...]
  D10  [+0.230, -0.034, +0.022, +0.021, -0.145, -0.018, ...]
  D11  [+0.011, +0.153, +0.025, +0.098, -0.060, +0.078, ...]
  D3   [-0.001, -0.282, -0.050, -0.142, -0.127, -0.070, ...]
  D4   [+0.096, -0.120, +0.062, -0.176, +0.141, -0.005, ...]
  D12  [-0.046, -0.035, +0.003, -0.163, -0.062, +0.156, ...]
  D13  [+0.065, -0.152, +0.063, -0.063, +0.062, +0.065, ...]
  D14  [+0.076, -0.202, +0.021, +0.055, +0.112, +0.029, ...]
  D15  [+0.005, -0.071, -0.165, -0.102, -0.052, +0.090, ...]
  D5   [-0.038, +0.122, +0.065, -0.037, +0.111, +0.218, ...]
  D6   [-0.117, +0.172, +0.029, +0.042, -0.049, -0.000, ...]
  D16  [+0.077, -0.130, +0.042, +0.058, -0.015, -0.048, ...]
  D17  [-0.182, +0.106, -0.006, +0.169, -0.107, +0.004, ...]
  D18  [-0.049, +0.246, -0.127, +0.039, -0.025, +0.045, ...]
  D19  [-0.188, +0.013, +0.210, -0.028, +0.158, -0.020, ...]
  D20  [-0.040, -0.098, +0.013, -0.004, +0.215, -0.053, ...]
  D21  [+0.107, -0.148, -0.027, +0.003, +0.043, -0.183, ...]
  D22  [-0.031, -0.007, -0.045, +0.075, +0.089, -0.248, ...]
  D23  [+0.026, -0.001, +0.179, -0.001, +0.049, +0.100, ...]
  D7   [-0.239, +0.125, -0.010, -0.075, -0.041, +0.012, ...]
  D24  [+0.105, -0.058, +0.038, -0.125, -0.087, +0.119, ...]

================================================================================
 Step 2: Anomaly Detection (mean similarity to 3 nearest neighbours)
================================================================================
Computed on the document vectors before the shared component is removed,
so a document unlike the rest of the collection stands out.

Median score = 0.864. Lowest scores:
  D7   0.567  A document about gardening  <-- ANOMALY
  D24  0.604  Football match report  <-- ANOMALY
  D12  0.826  Malware and antivirus software
  D10  0.837  Reinforcement learning
  D19  0.840  The TCP/IP protocol suite
  D17  0.846  Normalization
  ... (remaining documents score higher)
Threshold (median - 3*MAD) = 0.796
Detected anomalies: ['D7', 'D24']

================================================================================
 Step 3: Clustering (anomalies excluded)
================================================================================
Silhouette score (cosine) of K-means for different k (for reference):
  k = 2: 0.2301
  k = 3: 0.2236
  k = 4: 0.2142   <-- used
  k = 5: 0.1954
  k = 6: 0.1950
  k = 7: 0.1921
  k = 8: 0.2087
Silhouette scores are close for k = 3-5 on so few documents, so k = 4
is chosen to match the number of subject areas the notes come from.

K-means vs. hierarchical (average linkage) agreement, ARI = 0.727
Agreement with the true subject areas (used only for evaluation):
  K-means      : purity = 0.909, ARI = 0.748
  Hierarchical : purity = 0.909, ARI = 0.774

================================================================================
 Step 4: Topic Modelling (4 topics)
================================================================================

LDA topics (agreement of dominant topic with K-means clusters, ARI = 0.038):
  Topic 1: database, databases, records, relational, tables, data, traffic, store
  Topic 2: data, ip, networks, packets, internet, system, language, across
  Topic 3: models, network, language, virtual, layer, data, query, text
  Topic 4: networks, systems, scans, security, neural, models, access, learning

NMF topics (agreement of dominant topic with K-means clusters, ARI = 0.516):
  Topic 1: learning, networks, models, neural, training, deep, computer, vision
  Topic 2: databases, records, relational, database, sql, nosql, tables, queries
  Topic 3: networks, network, security, traffic, layer, private, routing, virtual
  Topic 4: language, natural, text, models, processing, chatbots, questions, word
```

```
LDA needs many more words than 22 short documents provide, so its topics
are unstable here. Cluster summaries below use NMF topics, which agree
better with the clusters.

================================================================================
 Step 5: Automatically Generated Clusters and Topic Summaries
================================================================================

Cluster 1 (4 documents): D19, D20, D21, D22
    D19  The TCP/IP protocol suite
    D20  Routing
    D21  Wireless networks
    D22  The OSI model
  Key terms      : routing, networks, tcp, packets, network, ip
  Dominant topic : NMF Topic 3 (networks, network, security, traffic, layer)
  Representative : D19 - The TCP/IP protocol suite

Cluster 2 (6 documents): D1, D2, D8, D9, D10, D11
    D1   Neural networks and deep learning
    D2   Natural language processing
    D8   Supervised machine learning
    D9   Computer vision
    D10  Reinforcement learning
    D11  Large language models and chatbots
  Key terms      : language, models, learning, large, machine, neural
  Dominant topic : NMF Topic 1 (learning, networks, models, neural, training)
  Representative : D2 - Natural language processing

Cluster 3 (5 documents): D3, D4, D12, D14, D15
    D3   Network security and firewalls
    D4   Cryptography and cyberattacks
    D12  Malware and antivirus software
    D14  Virtual private networks and TLS
    D15  Denial-of-service attacks
  Key terms      : traffic, service, malware, attacks, security, systems
  Dominant topic : NMF Topic 3 (networks, network, security, traffic, layer)
  Representative : D3 - Network security and firewalls

Cluster 4 (7 documents): D13, D5, D6, D16, D17, D18, D23
    D13  Authentication and passwords
    D5   Relational databases and SQL
    D6   Database indexing
    D16  Transactions and recovery
    D17  Normalization
    D18  NoSQL databases
    D23  Domain Name System
  Key terms      : database, databases, records, indexes, domain, name
  Dominant topic : NMF Topic 2 (databases, records, relational, database, sql)
  Representative : D5 - Relational databases and SQL

Unclustered anomalies: D7 (A document about gardening), D24 (Football match report)
  D7: best-fitting cluster is 2 with similarity -0.010 (members average 0.627)
  D24: best-fitting cluster is 2 with similarity 0.068 (members average 0.627)

================================================================================
 Step 6: 2D Visualisation (PCA and t-SNE)
================================================================================
PCA of the raw document vectors (the space used for anomaly detection):
  explained variance PC1 = 0.200, PC2 = 0.130
  PC1 separates computing documents from the unrelated ones.
t-SNE of the clustering vectors (shared component removed) shows the clusters.
Saved: pca_clusters.png, tsne_clusters.png, dendrogram.png

================================================================================
 Step 7: Search Results for "documents about protecting computer networks"
================================================================================
Query words: ['protecting', 'computer', 'networks']
Closest cluster: Cluster 3 (similarity 0.452) -> searched first

Cluster-based semantic search:
Rank Doc    Score  Cluster  Title / Why it matches
--------------------------------------------------------------------------------
1    D12    0.810  C3       Malware and antivirus software
                           matches: computer~computers (0.88), protecting~protect (0.84), networks~systems (0.66)
2    D14    0.761  C3       Virtual private networks and TLS
                           matches: networks~networks (1.00), protecting~preventing (0.70), computer~user (0.68)
3    D3     0.709  C3       Network security and firewalls
                           matches: networks~networks (1.00), computer~systems (0.72), protecting~security (0.55)
4    D4     0.667  C3       Cryptography and cyberattacks
                           matches: computer~digital (0.70), protecting~protects (0.67), networks~digital (0.61)
5    D15    0.617  C3       Denial-of-service attacks
                           matches: computer~online (0.67), networks~online (0.60), protecting~limiting (0.59)
```

```
Score = weighted mean similarity between each query word and its closest
word in the document (shown as query~document (cosine)).
Retrieved without containing any exact query word: D12, D4, D15

Semantic search over all documents:
  1. D12  0.810  Malware and antivirus software
  2. D14  0.761  Virtual private networks and TLS
  3. D9   0.750  Computer vision
  4. D3   0.709  Network security and firewalls
  5. D21  0.668  Wireless networks

Keyword (TF-IDF) baseline:
  1. D9   0.346  Computer vision
  2. D21  0.197  Wireless networks
  3. D1   0.164  Neural networks and deep learning
  4. D3   0.119  Network security and firewalls
  5. D20  0.066  Routing

================================================================================
 Step 8: Retrieval Evaluation (top 5)
================================================================================

Q1: "documents about protecting computer networks"   relevant = ['D3', 'D4', 'D12', 'D14', 'D15']
  Semantic        ['D12', 'D14', 'D9', 'D3', 'D21']  P@5 = 0.60  R@5 = 0.60  AP = 0.55
  Doc-vector      ['D14', 'D3', 'D21', 'D12', 'D4']  P@5 = 0.80  R@5 = 0.80  AP = 0.71
  Cluster-based   ['D12', 'D14', 'D3', 'D4', 'D15']  P@5 = 1.00  R@5 = 1.00  AP = 1.00
  Keyword TF-IDF  ['D9', 'D21', 'D1', 'D3', 'D20']  P@5 = 0.20  R@5 = 0.20  AP = 0.05

Q2: "how do computers understand human language"   relevant = ['D2', 'D11']
  Semantic        ['D2', 'D11', 'D9', 'D23', 'D1']  P@5 = 0.40  R@5 = 1.00  AP = 1.00
  Doc-vector      ['D2', 'D11', 'D9', 'D10', 'D12']  P@5 = 0.40  R@5 = 1.00  AP = 1.00
  Cluster-based   ['D2', 'D11', 'D9', 'D1', 'D10']  P@5 = 0.40  R@5 = 1.00  AP = 1.00
  Keyword TF-IDF  ['D2', 'D11', 'D12', 'D23']  P@5 = 0.40  R@5 = 1.00  AP = 1.00

Q3: "making database queries run faster"   relevant = ['D5', 'D6', 'D18']
  Semantic        ['D18', 'D6', 'D5', 'D23', 'D17']  P@5 = 0.60  R@5 = 1.00  AP = 1.00
  Doc-vector      ['D18', 'D5', 'D23', 'D6', 'D19']  P@5 = 0.60  R@5 = 1.00  AP = 0.92
  Cluster-based   ['D18', 'D6', 'D5', 'D23', 'D17']  P@5 = 0.60  R@5 = 1.00  AP = 1.00
  Keyword TF-IDF  ['D5', 'D16', 'D6', 'D18', 'D23']  P@5 = 0.60  R@5 = 1.00  AP = 0.81

Q4: "how data travels across the internet"   relevant = ['D19', 'D20', 'D21', 'D22', 'D23']
  Semantic        ['D19', 'D20', 'D18', 'D12', 'D21']  P@5 = 0.60  R@5 = 0.60  AP = 0.52
  Doc-vector      ['D21', 'D20', 'D18', 'D15', 'D14']  P@5 = 0.40  R@5 = 0.40  AP = 0.40
  Cluster-based   ['D19', 'D20', 'D21', 'D22', 'D12']  P@5 = 0.80  R@5 = 0.80  AP = 0.80
  Keyword TF-IDF  ['D19', 'D20', 'D18', 'D4', 'D23']  P@5 = 0.60  R@5 = 0.60  AP = 0.52

Q5: "teaching machines to learn from examples"   relevant = ['D1', 'D8', 'D9', 'D10']
  Semantic        ['D8', 'D9', 'D1', 'D2', 'D10']  P@5 = 0.80  R@5 = 1.00  AP = 0.95
  Doc-vector      ['D11', 'D2', 'D9', 'D8', 'D10']  P@5 = 0.60  R@5 = 0.75  AP = 0.36
  Cluster-based   ['D8', 'D9', 'D1', 'D2', 'D10']  P@5 = 0.80  R@5 = 1.00  AP = 0.95
  Keyword TF-IDF  ['D8', 'D9', 'D1', 'D15']  P@5 = 0.60  R@5 = 0.75  AP = 0.75

Method               P@5     R@5    F1@5     MRR     MAP
--------------------------------------------------------
Semantic           0.600   0.840   0.682   1.000   0.804
Doc-vector         0.560   0.790   0.638   0.867   0.677
Cluster-based      0.720   0.960   0.802   1.000   0.950
Keyword TF-IDF     0.480   0.710   0.558   0.850   0.625

================================================================================
 Summary
================================================================================
* Embeddings: GloVe word vectors combined into SIF document vectors let
  the system match related words (protecting~protect, networks~systems).
* Anomalies: D7 (A document about gardening), D24 (Football match report)
  were flagged because they are far from every other document.
* Clustering: K-means found 4 clusters with purity 0.91
  against the true subjects. Misplaced documents share vocabulary with
  another subject, e.g. DNS (servers, query, cache) with databases.
* Topics: NMF gave clear themes, but LDA was unstable on a corpus this small.
* Retrieval: Cluster-based search had the best MAP (0.950) vs.
  0.625 for keyword TF-IDF. Restricting the search to the
  closest cluster removes off-topic documents that share a single word
  with the query (e.g. "Computer vision" for "computer networks").
(.venv) PS D:\SVNIT\Semester - 7\Information Retrieval\Tutorials\Tutorial - 5>
```